

openMSP430 Microcontroller Design and Optimization

CS 220 Synthesis of Digital Systems

Team 5 Project Proposal

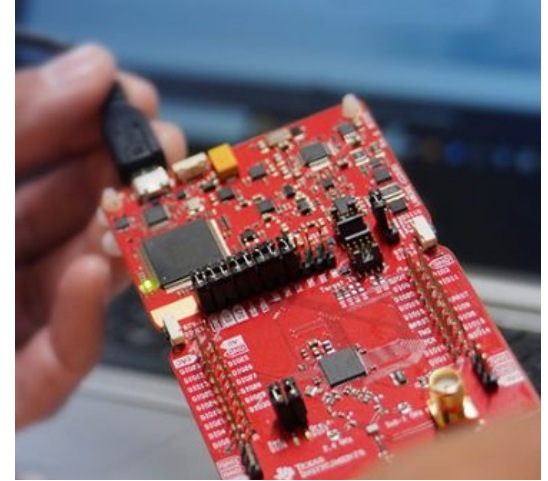
Andrew Permatigari, Kenneth Maina, Xingran Huang

openMSP430 Overview

openMSP430 is an open-source, synthesizable 16-bit microcontroller core implemented in Verilog HDL, fully compatible with the Texas Instruments MSP430 architecture.

Key characteristics:

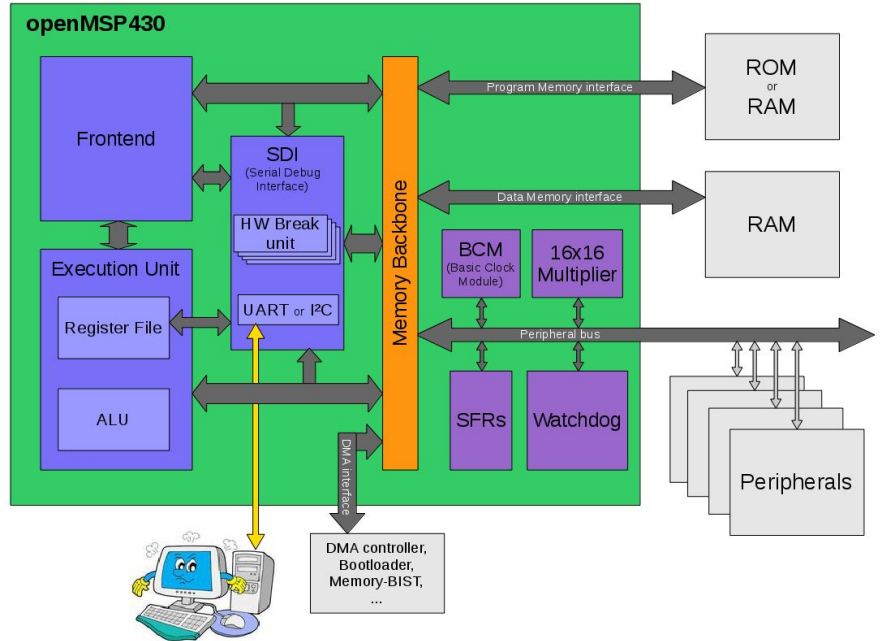
- Compact silicon footprint: approximately 8,000 logic gates
- Supports full MSP430 instruction set with near cycle-accurate execution
- Integrated peripherals: 16x16 Hardware Multiplier, Basic Clock Module, Watchdog Timer, GPIO, Timer A
- Proven in both FPGA and ASIC implementations
- Licensed under BSD, actively maintained open-source community
- Built-in Low Power Modes (LPM0 to LPM4) and clock management support



Architecture Overview

The openMSP430 core consists of the following key subsystems:

- Execution Unit: handles instruction decode, ALU operations, and register file management
- Memory Interface: configurable program and data memory with scalable address space
- Clock and Power Management: Basic Clock Module supporting multiple low-power operating modes
- Peripheral Bus: connects integrated peripherals including multiplier, watchdog, and GPIO
- Debug Interface: two-wire Serial Debug Interface with GDB support for in-system debugging



Proposed Optimization: Clock Gating

Clock gating is a widely used low-power design technique that disables the clock signal to inactive logic blocks, eliminating unnecessary switching activity and reducing dynamic power consumption.

Why Clock Gating for openMSP430?

- The openMSP430 already includes a Basic Clock Module and built-in Low Power Mode support, providing a natural foundation for clock gating implementation.
- Targeted modules include the Hardware Multiplier, Watchdog Timer, and peripheral logic, which are frequently idle during normal operation.

Expected Outcome:

- Measurable reduction in dynamic power consumption with minimal impact on area and timing performance.

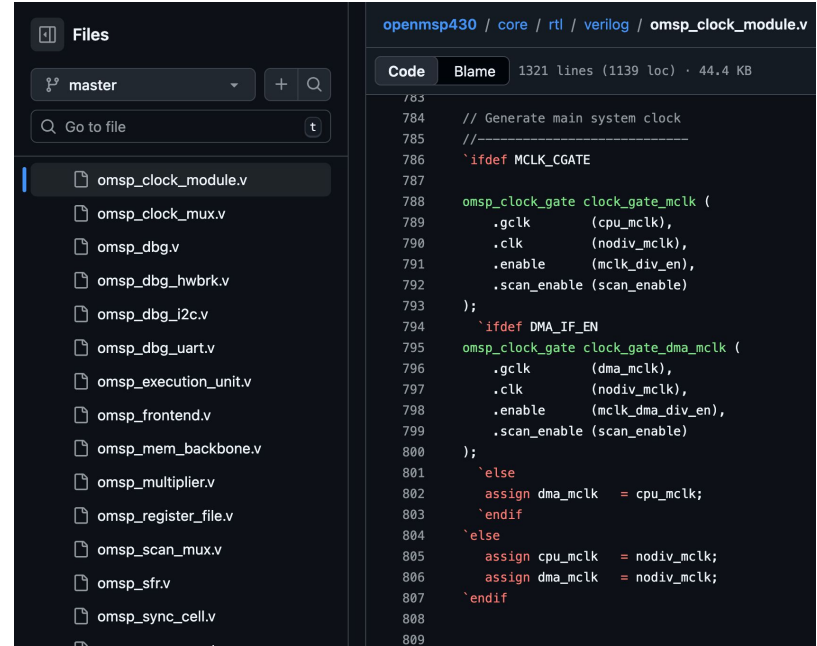
Methodology

Our approach follows a structured baseline-then-optimize workflow:

Step 1: Baseline Synthesis Synthesize the original openMSP430 RTL design using industry-standard EDA tools to obtain baseline metrics including area, dynamic power, and timing.

Step 2: Clock Gating Implementation Identify idle logic blocks within the design and introduce clock gating cells at the RTL level to suppress unnecessary clock toggling.

Step 3: Post-Optimization Synthesis and Comparison Re-synthesize the modified design and compare power, area, and timing results against the established baseline to quantify the impact of the optimization.



```
Files
master
Go to file
omsp_clock_module.v
omsp_clock_mux.v
omsp_dbg.v
omsp_dbg_hwbrk.v
omsp_dbg_i2c.v
omsp_dbg_uart.v
omsp_execution_unit.v
omsp_frontend.v
omsp_mem_backbone.v
omsp_multiplier.v
omsp_register_file.v
omsp_scan_mux.v
omsp_sfr.v
omsp_sync_cell.v

openmsp430 / core / rtl / verilog / omsp_clock_module.v
Code Blame 1321 lines (1139 loc) · 44.4 KB
783
784 // Generate main system clock
785 //-----
786 `ifndef MCLK_CGATE
787
788 omsp_clock_gate clock_gate_mclk (
789     .clk      (cpu_mclk),
790     .clk      (nodiv_mclk),
791     .enable   (mclk_div_en),
792     .scan_enable (scan_enable)
793 );
794 `ifdef DMA_IF_EN
795 omsp_clock_gate clock_gate_dma_mclk (
796     .clk      (dma_mclk),
797     .clk      (nodiv_mclk),
798     .enable   (mclk_dma_div_en),
799     .scan_enable (scan_enable)
800 );
801 `else
802 assign dma_mclk = cpu_mclk;
803 `endif
804 `else
805 assign cpu_mclk = nodiv_mclk;
806 assign dma_mclk = nodiv_mclk;
807 `endif
808
809
```

Weekly Timeline

Week 3-4: Deeper dive into openMSP430 code and documentation; set up EDA synthesis tools

Week 5: Focus on clock and power management modules and identify targets for clock gating

Week 6: Run baseline synthesis; validate results; submit midterm report

Week 7: Begin RTL modification for clock gating implementation

Week 8: Complete clock gating implementation; run post-optimization synthesis

Week 9: Analyze and compare results; prepare final report

Week 10: Final presentation and report submission

Thank you

Reference :

<https://opencores.org/projects/openmsp430/core>

<https://www.ti.com/design-development/embedded-development/msp430-mcus.html>

<https://github.com/olgirard/openmsp430>